

1 Data Pipeline, Baseline und Experiment-Tracking

Jan-David Wiederstein

Aus den Befunden des Data Understanding ist eine gemeinsame Pipeline entstanden. Sie liefert allen Ansätzen dieselben bereinigten Daten, denselben Split, dieselbe Vorverarbeitung und dieselben Metriken – Ergebnisse unterschiedlicher Modelle sind damit direkt vergleichbar. Das Modell selbst baut jeder Ansatz eigenständig (Abb. 1).

Was die Pipeline macht

- **Datenaufbereitung** (`make data`): Rohdaten werden per Schema validiert, zwei exakt doppelte Zeilen entfernt (sonst könnte dieselbe Messung in Training *und* Validierung landen). Daraus entstehen ein fester 70/30-Holdout und fünf CV-Folds im Trainingsteil, stratifiziert nach `Crop|Stage`, damit auch seltene Paare wie `cotton|Harvest` (11 Zeilen) in beiden Teilen vorkommen – per festem Seed bei allen identisch.
- **Vorverarbeitung** als `sklearn-Pipeline`, die nur auf dem Trainingsteil gefittet wird: 67 leere Bänder entfernen (131 bleiben), Lücken entlang der Wellenlänge interpolieren, `AEZ/Month` sowie Skalierung und Vegetationsindizes optional zuschaltbar.
- **Bewertung**: Balanced Accuracy, Macro-F1 und Samples-F1 für Crop, Stage und kombiniert sowie der Anteil ungültiger Crop/Stage-Paare.

Verwendung

Hyperparameter werden mit `tune()` ausschließlich per Cross-Validation auf dem Trainingsteil gesucht; das beste Modell wird mit `run()` genau einmal auf dem Holdout bewertet. So wird nicht unbemerkt auf den Validierungsdaten optimiert.

```
tuned = tune(model, TuneConfig(name=..., approach=..., param_grid={...}))
result = run(tuned.best_model, RunConfig(name=..., tuning_run=tuned.run_id))
```

Baseline

Modell	BAcc Crop	BAcc Stage	BAcc kombiniert	Samples-F1
Dummy (häufigste Klasse)	0,200	0,167	0,044	0,326
Random Forest (300 Bäume, balanced)	0,890	0,870	0,743	0,878

Der Random Forest liegt mit 0,743 im Streubereich seines CV-Scores ($0,750 \pm 0,019$), überschätzt sich also nicht. Verwechselt werden vor allem Mais/Soja und benachbarte Stadien; 0,3 % der Vorhersagen sind unmögliche Paare. Jeder weitere Ansatz muss 0,743 schlagen.

Wie MLflow uns hilft

Jeder Aufruf von `tune()` und `run()` wird automatisch protokolliert: Parameter, Metriken, Confusion Matrices, trainiertes Modell, Hash der Daten und Git-Stand. So lassen sich Läufe in der Oberfläche (`make mlflow`) nach Ansatz gruppieren und vergleichen, jedes Ergebnis ist auf Code und Daten zurückführbar, und gespeicherte Modelle können direkt für die Testvorhersage geladen werden. Wichtige Läufe kommen mit Run-ID ins Experiment-Log.

Was noch fehlt

- Weitere Vorverarbeitung als Optionen: Glättung (z. B. Savitzky-Golay), PCA oder Bandauswahl, zyklische Kodierung von `Month`, Umgang mit Ausreißern.
- Nur gültige Crop/Stage-Kombinationen vorhersagen (z. B. hierarchisch); gruppierter Split, falls sich räumliche Cluster bestätigen; Vorhersage für `test.csv`.
- MLflow läuft lokal (jede:r sieht nur eigene Läufe) – ein gemeinsamer Server würde nur das Tracking-Modul betreffen.

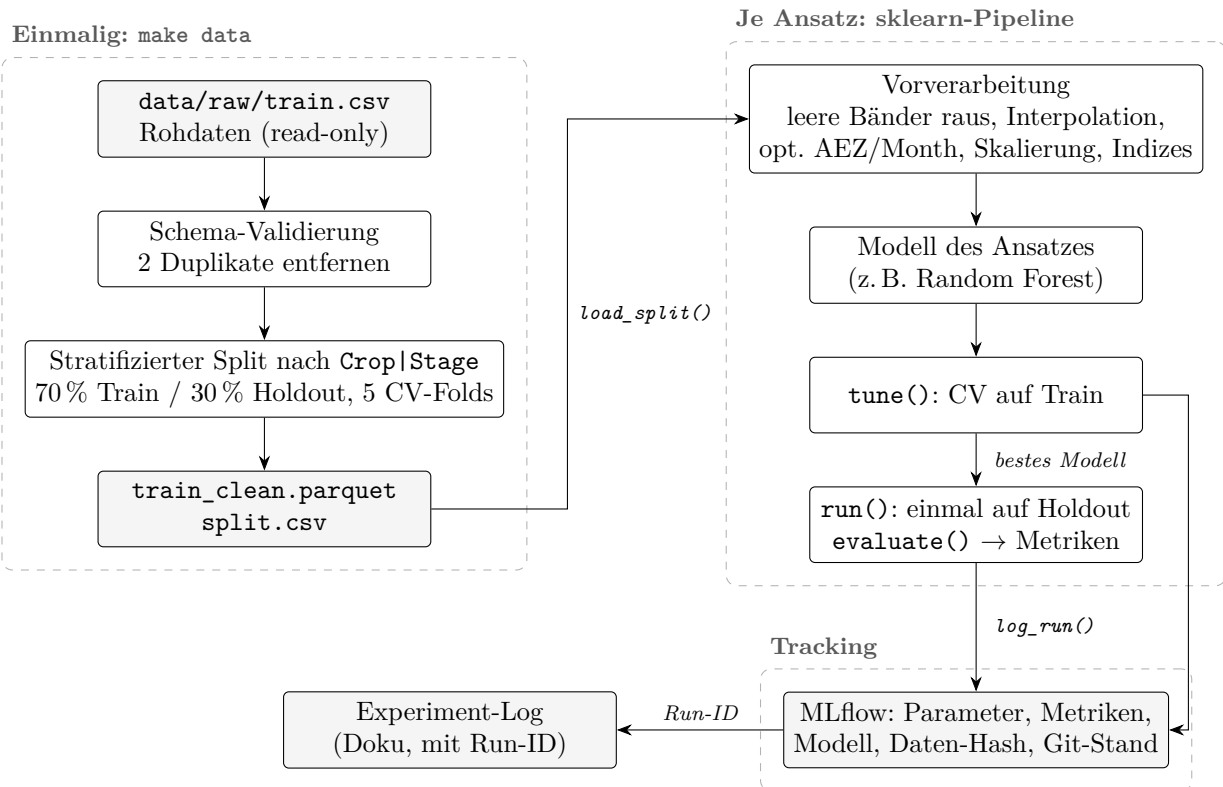


Abbildung 1: Gemeinsame Data Pipeline: Die Datenaufbereitung läuft einmal und ist für alle gleich; Vorverarbeitung und Modell werden je Ansatz nur auf dem Trainingsteil gefittet und jeder Lauf in MLflow protokolliert.